



Article

iKern: Advanced Intrusion Detection and Prevention at the Kernel Level Using eBPF

Hassan Jalil Hadi ^{1,*}, Mubashir Adnan ^{2,*}, Yue Cao ¹, Faisal Bashir Hussain ², Naveed Ahmad ³, Mohammed Ali Alshara ^{3,4} and Yasir Javed ³

¹ School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China; yue.cao@whu.edu.cn;

² Department of Computer Science, Bahria University, Islamabad 44220, Pakistan; fbashir.buic@bahria.edu.pk

³ College of Computer and Information Sciences, Prince Sultan University, Riyadh 11586, Saudi Arabia; nahmed@psu.edu.sa (N.A.); malshara@psu.edu.sa (M.A.A.); yjaved@psu.edu.sa (Y.J.)

⁴ College of Computer Sciences and Information for Educational and Quality Affairs, Al-Imam Muhammad Ibn Saud Islamic University, Riyadh 11432, Saudi Arabia

* Correspondence: hjhwhu@whu.edu.cn (H.J.H.); madnan.buic@bahria.edu.pk (M.A.)

[†] These authors contributed equally to this work.

Abstract: The development of new technologies has significantly enhanced the monitoring and analysis of network traffic. Modern solutions like the Extended Berkeley Packet Filter (eBPF) demonstrate a clear advancement over traditional techniques, allowing for more customized and efficient filtering. These technologies are crucial for influencing system performance as they operate at the lowest layer of the operating system, such as the kernel. Network-based Intrusion Detection/Prevention Systems (IDPS), including Snort, Suricata, and Bro, passively monitor network traffic from terminal access points. However, most IDPS are signature-based and face challenges on large networks, where the drop rate increases due to limitations in capturing and processing packets. High throughput leads to overheads, causing IDPS buffers to drop packets, which can pose serious threats to network security. Typically, IDPS are targeted by volumetric and multi-vector attacks that overload the network beyond the reception and processing capacity of IDPS, resulting in packet loss due to buffer overflows. To address this issue, the proposed solution, iKern, utilizes eBPF and Virtual Network Functions (VNF) to examine and filter packets at the kernel level before forwarding them to user space. Packet stream inspection is performed within the iKern Engine at the kernel level to detect and mitigate volumetric floods and multi-vector attacks. The iKern detection engine, operating within the Linux kernel, is powered by eBPF bytecode injected from user space. This system effectively handles volumetric Distributed Denial of Service (DDoS) attacks. Real-time implementation of this scheme has been tested on a 1Gbps network and shows significant detection and reduction capabilities against volumetric and multi-vector floods.

Keywords: eBPF; intrusion prevention system (IPS); intrusion detection system (IDS); virtual network functions (VNF); kernel-level packet filtering; DDoS



Citation: Hadi, H.J.; Adnan, M.; Cao, Y.; Hussain, F.B.; Ahmad, N.; Alshara, M.A.; Javed, Y. iKern: Advanced Intrusion Detection and Prevention at the Kernel Level Using eBPF.

Technologies **2024**, *12*, 122.

<https://doi.org/10.3390/technologies12080122>

Received: 2 July 2024

Revised: 25 July 2024

Accepted: 26 July 2024

Published: 30 July 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The eBPF is a powerful in-kernel virtual machine prominently utilized for network packet filtering and system monitoring [1]. eBPF functions by employing kernel hooks that modify certain kernel behaviours for various applications. Among these hooks is the Express Data Path (XDP) [2], which enables the attachment of eBPF programs directly to the network device drivers. This capability allows for the early processing of packets through tailored programming, optimizing performance and flexibility.

The Extended Berkeley Packet Filter (eBPF) [3] presents numerous benefits, distinguishing it from other kernel bypass techniques such as the Data Plane Development Kit (DPDK) [4]. Firstly, eBPF integrates seamlessly with the existing network stack, allowing

developers to selectively leverage pre-existing data structures or functions. This integration significantly reduces development time and effort. Secondly, eBPF excels in performance for data plane applications, thanks to its ability to process packets at an early stage via the XDP. This early processing minimizes the overhead typically encountered in higher network stack layers. Lastly, unlike DPDK—which requires dedicating entire CPU cores to packet processing and operates in polling mode often leading to underutilized computing resources during low traffic—eBPF operates on an interrupt-based mechanism. This approach ensures that CPU usage is efficiently scaled according to the traffic load, offering improved performance and CPU power efficiency across various network applications.

An Intrusion Detection System (IDS) monitors network traffic to detect suspicious activity [5,6] and issues alerts when such activity is detected [7]. Essentially, it is a software application that scans a network or system either fully or partially and reports any malicious activities. Any violations or breaches of policy are typically reported either to a central administrator or collected centrally through a Security Information Event Management system (SIEM) [8]. Some IDSs are capable of responding immediately to a detected intrusion. These systems are classified as Intrusion Prevention Systems (IPS) [9]. These traffic monitoring applications, along with other tools like tcpdump, Wireshark, tshark, and netsniff-ng, perform traffic processing and signature matching in the user space of an operating system [10]. They utilize pattern-matching algorithms, typically written in C or Python, to detect malicious packets that could compromise the system or network. These user space detection tools can identify and mitigate attacks based on predefined signatures. However, the packet capturing process requires the operating system kernel to copy packets from the Network Interface Card (NIC) to data acquisition buffers before forwarding them to user space. While packet capturing in the kernel space is fast, increasing throughput from 300 Mbps to 500 Mbps presents challenges such as buffer overflows, packet drops, and increased latency [11,12].

Due to these vulnerabilities within networks, attackers have developed innovative methods for exploiting network traffic monitoring applications. Recent studies [1,13–15] indicate that while traffic monitoring solutions like IDS and IPS effectively detect and mitigate contemporary attacks, these systems can still be compromised by multi-vector and volumetric flood attacks. These modern attacks combine elements of traditional DoS and DDoS attacks into a hybrid form [16]. For example, multiple DDoS tactics might be merged to create a single attack vector, such as combining a TCP flood with a SYN flood attack to form a unified hybrid vector [17]. This type of attack is executed at varying data rates, specifically to exploit the limitations of packet-capturing buffers, causing them to overflow at their maximum thresholds and rendering the IDS or IPS systems ineffective, leaving the network vulnerable.

Furthermore, IDPS are essential for detecting and mitigating cyber-attacks through either pattern-matching or anomaly-based techniques [18]. Typically, these systems operate their detection engines in user space, which significantly increases CPU overhead when processing large network flows. Volumetric and multi-vector DDoS attacks exploit the vulnerabilities of both IDPS detection engines and data acquisition modules [19,20]. These attacks intensify CPU overheads and lead to buffer overflows by routing excessive packets from kernel space to user space detection engines. Consequently, effectively detecting volumetric and multi-vector flood attacks at the kernel level to reduce network traffic overhead and protect system resources from exploitation poses a significant challenge.

In response to these challenges, this study introduces the iKern system, engineered to withstand multi-vector and volumetric flood attacks. The iKern Data Acquisition Module is crafted using PF_RING sockets to prevent buffer overflows even under large data flows [21]. This module employs multi-core packet capturing across multiple iKern instances, implementing load balancing to distribute incoming traffic evenly and avoid CPU overhead. To identify and mitigate multi-vector and volumetric floods, a lightweight In-kernel Detection Algorithm (IDA) inspects each incoming packet. Malicious packets are dropped before

they are forwarded to user space, utilizing in-kernel eBPF rules to ensure effective filtering. This study offers several significant contributions, including the following.

- An innovative detection and mitigation framework, iKern, designed to intercept and analyze high-throughput network traffic up to 1 Gbps using eBPF for dynamic packet filtering at the kernel level, substantially reducing the overhead associated with traditional user-space systems.
- The development of an optimized data acquisition module that employs PF_RING and eBPF to efficiently distribute network load across multiple processor cores, minimizing CPU overhead and enhancing throughput, ensuring robust real-time processing under heavy network loads.
- Extensive testing and evaluation of the iKern system to demonstrate its effectiveness in improving detection rates and overall system performance, particularly in mitigating volumetric and multi-vector attacks, thus establishing a new standard for intrusion detection and prevention technologies in terms of efficiency and scalability.

This research paper is organized as follows: Section 2 reviews the existing literature related to eBPF-based IDPS. Section 3 explains the development and implementation of iKern-IDPS, along with experimental evaluation and discussion. Section 4 presents the real-time implementation and performance analysis. Lastly, our findings are concluded in Section 5.

2. Related Work

Various attack detection and prevention mechanisms based on both user space and kernel space detection models have been proposed. Signature and anomaly-based IDSs identify malicious traffic using detection engines that operate in the user space of an operating system. Kernel space detection models, on the other hand, require the creation of VNFs and the injection of bytecode from user space to render the kernel programmable for executing customized functions using eBPF. This literature review discusses attack detection models based on user space IDS and kernel space attack detection leveraging VNFs. The existing body of work in this domain can be broadly categorized into two models: the Kernel Space Detection Model (KDM) and the User Space Detection Model (UDM).

Additionally, eBPF has found extensive application across various data plane contexts. Vieira et al. [22] conducted an exhaustive survey on eBPF, detailing both the technical aspects and the fields in which eBPF is actively employed. eBPF has revolutionized various basic network functions, including switching [23], routing [24], and firewall operations [25]. Its advanced capabilities have facilitated the development of complex applications such as load balancing, key-value storage, application-level filtering, and DDoS mitigation [1]. Particularly notable is the surge in eBPF-related research within the NFV space. For instance, Ahmed et al. [26] introduced In-KeV, an in-kernel framework designed to construct network function chains using eBPF's tail call feature. Moreover, Miano et al. [27] explored eBPF's limitations and shared their practical experiences, while assessing the performance of eBPF in packet filtering based on the five-tuple information from packet headers. With the rise of 5G networks, Parola et al. [28] developed an open-source 5G mobile gateway leveraging eBPF. Additionally, they implemented eBPF for monitoring traffic between virtual machines. Miano et al. [14] proposed a framework for deploying eBPF-based network functions within microservice environments.

Moreover, Xu et al. [29] explored the optimization of eBPF bytecode through the development of K2, a specialized compiler for program synthesis that enhances eBPF bytecode. Their findings revealed significant improvements, including bytecode size reductions between 6–26%, an increase in packet processing throughput by up to 4.75% per core, and reductions in average packet processing latency ranging from 1.36% to 55.03%. K2 operates independently from the eBPF static verifier, utilizing dual sets of checks [30]. It generates multiple versions of bytecode, which are then submitted to the eBPF verifier. Versions not approved by the verifier are discarded, and the performance of the remaining ones is analyzed to assess differences in latency, size, and throughput. However, one

limitation of K2 is that it does not integrate directly with the eBPF runtime optimizations, which can limit its applicability in dynamic network environments. This is a gap that this study addresses by proposing a more integrated and context-aware optimization approach.

In the domain of packet processing and attack mitigation, recent studies have highlighted innovative applications of eBPF and XDP. Marcos A. M. Vieira [31] introduced a robust packet processing solution using eBPF and XDP, focusing on traffic filtering based on the TCP protocol. This study elaborated on the mechanics of BPF and eBPF and provides a comprehensive overview of the eBPF system within the Linux kernel, emphasizing recent advancements and applications. Vieira developed a program that enhances network monitoring performance by utilizing eBPF to establish TCP-specific filters, effectively dropping packets lacking TCP components to counter TCP reset attacks. The work is showcased through two perspectives: the higher-level C code and the original eBPF code post-compilation, highlighting its direct integration with the XDP hook. Building on this foundation, Cyril Cassagnes [32] proposed a DDoS attack detection architecture that employs eBPF and XDP functions. This architecture utilizes the Linux subsystem to oversee containerized user space applications, which aids in the exploration and utilization of the eBPF tool landscape. The principal contribution of Cassagnes' study revolves around using an eBPF/XDP program specifically designed to detect and mitigate DDoS attacks by operating on libpcap. This program preemptively identifies and filters attack traffic, effectively dropping harmful packets before they can reach user space and thus preventing CPU overheads.

Moreover, Josy Elsa Varghese [33] proposed a framework for an IDS within Software-Defined Networking (SDN) environments that uses a statistical parameter linked to the DPDK framework [34]. This framework efficiently addresses the challenges of high-speed network environments and the limitations of conventional IDS [35]. The system, tested using the CICDDoS2019 dataset, demonstrated a remarkable balance between detection efficiency and effectiveness, achieving an accuracy of 96.59% at a throughput of 720 Mbps.

Building on different aspects of network security, Sumit Badotra [36] developed a DDoS detection system using SNORT IDS in the Opendaylight (ODL) and Open Networking Operating System frameworks. This system was rigorously tested under various scenarios with different numbers of hosts, switches, and traffic volumes generated using tools like hping3 and nping. The system's performance was evaluated based on packet metrics and CPU utilization, showcasing its robustness in diverse network settings.

Further extending IDS research, [37] introduced a novel method that combines ML and statistical techniques to detect both low- and high-volume DDoS attacks. The method utilizes an entropy-based collector alongside a classification system, which was critically evaluated using UNB-ISCX1 and CTU-132 datasets. The results indicated an exceptionally high accuracy of 99.85% for volumetric attacks at a network throughput of 500 Mbps, highlighting the potential of integrating diverse analytical techniques in IDS.

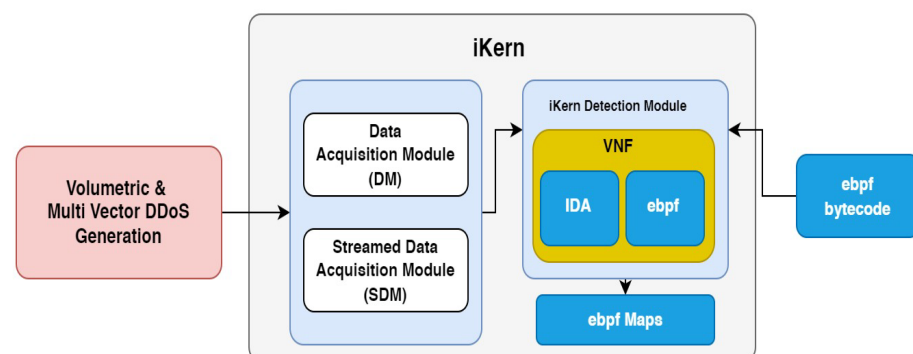
Additionally, it has been observed that Kernel Space Attack Detection Models (KDMs) significantly outperform User Space Attack Detection Models (UDMs) in terms of volumetric attack detection accuracy, packet reception rate, packet drop rate, and CPU utilization. However, UDM systems demonstrate better accuracy on average-sized networks, although CPU overhead and packet drop rates increase on larger networks. Detection engines operating in user space require more processing power compared to Virtual Network Functions (VNFs) running within kernel space. Packets received by the network card are copied to the kernel where attacks are detected before the flow is forwarded to other userland applications, thereby reducing CPU overhead. A comparison of related work based on threat detection accuracy, data acquisition methods, and network throughput achieved is shown in Table 1.

Table 1. Comparative analysis of related work by type, network throughput, and threat detection accuracy.

Ref.	eBPF	DAQ	Type	Throughput CPU		Accuracy	Scale	Time	Level
[14]	✓	afpacket	KDM	✓	×	✓	×	✓	Kernel
[28]	✓	libpcap	KDM	×	×	×	×	×	Kernel
[15]	✓	libpcap	KDM	×	✓	✓	✓	✓	Kernel
[17]	×	dpdk	UDM	✓	✓	×	✓	×	User
[21]	×	afpacket	UDM	×	×	✓	×	×	User
[33]	✓	libpcap	KDM	×	×	✓	×	×	Kernel
[36]	×	libpcap	UDM	×	×	×	×	×	User
[37]	×	libpcap	UDM	×	×	✓	×	×	User
Our Work	✓	PF-Ring	KDM	✓	✓	✓	✓	✓	Kernel

3. Development and Implementation of iKern-IDPS

The proposed system consists of distinct modules as depicted in Figure 1. The DM is engineered to capture packets across extensive network flows. To assess the capabilities of the DM, both volumetric and multi-vector floods are generated. A multi-core implementation enhances the DM within the Streamed Data Acquisition Module (SDM) through the application of load balancing techniques. Traffic inspection is conducted by an in-kernel detection module, which utilizes eBPF actions and bytecode injected from user space, thereby creating a VNF.

**Figure 1.** Schematic diagram of the iKern-IDPS system illustrating the main components and data flow.

3.1. Volumetric Multi Vector Attack Generation

For evaluating the performance of the proposed system, the CIC-DDoS2019 [38,39] dataset is utilized. This dataset includes both benign data and recent common volumetric DDoS attacks, accurately mirroring real-world scenarios in the form of pcap files [40]. The included attacks comprise UDP, TCP, PortMap, NetBIOS, LDAP, MSSQL, SYN, DNS, NTP, and SNMP floods. In addition to this dataset, the evaluation also incorporates some of the most widely used and effective tools for launching volumetric and multi-vector attacks, such as HIOC, RUDY, HULK, and LOIC. These tools are known for their ability to generate large volume flows, significantly straining the resources of the targeted systems. These tools were employed to assess the data acquisition and attack detection capabilities of the proposed system against large-scale, high-rate attacks.

3.2. Data Acquisition Module (DM)

The DM is essential for receiving network traffic from high-speed networks or industrial systems. As technological advancements have expanded the use of large-scale

networks in corporate sectors, these networks demand high-performance hardware and software. This includes modern network cards equipped with multiple read/write queues, multi-core CPUs, data distribution algorithms, and load-balancing techniques. The default network stacks provided by operating systems like Linux and Windows, which utilize standard libraries for packet capturing, tend to drop packets when dealing with large networks and high data rates, as highlighted in the comparative analysis section. Deploying a traffic monitoring solution in corporate environments requires robust data acquisition and processing capabilities. The proposed solution aims to intercept network packets from substantial flows without dropping them, as packet loss can pose a serious security risk to organizations or companies. To ensure efficient operation, a High-Speed Data Acquisition Unit has been implemented to capture packets from large networks effectively.

The DM is specifically designed to integrate the PF_RING kernel module into the proposed architecture. DM leverages PF_RING to efficiently acquire traffic from high-speed networks. To minimize the cost associated with system calls for each packet, DM utilizes PF_RING to establish a ring buffer in the kernel space, which is then shared with the iKern Engine. iKern accesses this kernel space buffer through NAPI polling. Initially, the user space application performs a single system call to access the kernel space ring buffer, retrieves packets from this buffer, and processes them until the buffer is depleted. Once there are no more packets left in the buffer for processing, iKern makes another system call upon the arrival of new packets in the buffer. This approach significantly reduces the overhead associated with individual system calls for each packet reception, enabling iKern to process a higher volume of packets more efficiently. The operations of PF_RING are directly managed by DM, and the creation of the ring buffer using the PF_RING socket is depicted in Figure 2.

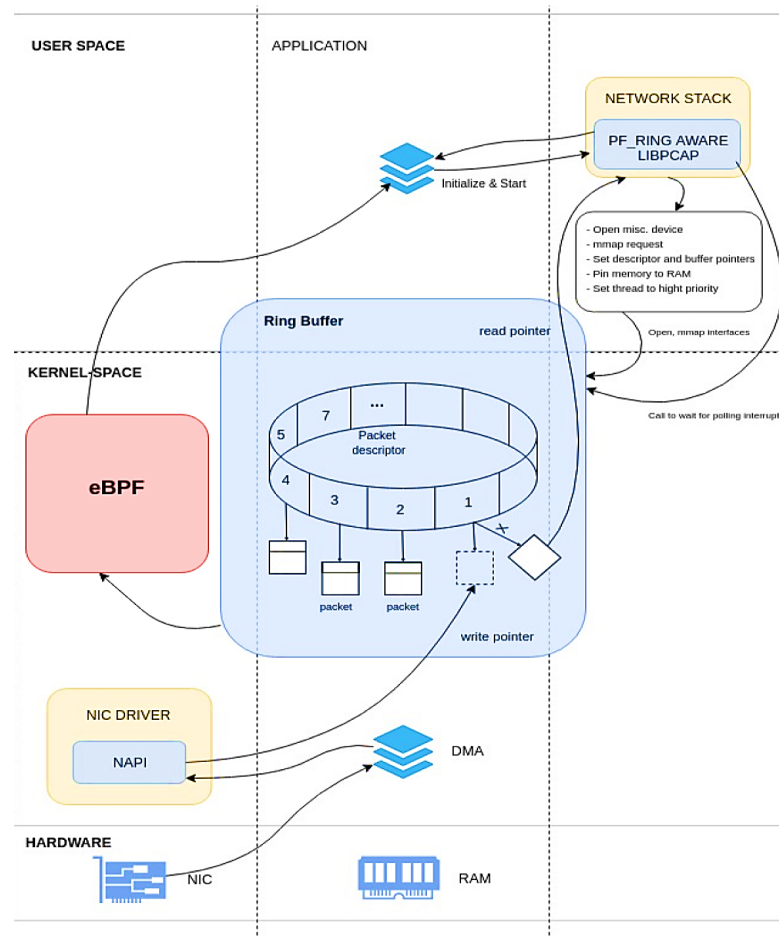


Figure 2. DM using PF RING socket.

The DM activates the PF_RING socket via the `pfring_daq_open()` and `pfring_open()` functions, which are instrumental in allocating a ring buffer. This buffer is deallocated upon the deactivation of the socket. When a packet arrives at the NIC, it is first captured by the device driver and then transferred to the PF_RING socket using the `pfring_daq_acquire()` and `pfring_recv()` functions. If the ring buffer reaches capacity, incoming packets may be rejected. Packets that successfully enter the ring buffer are subsequently forwarded to the eBPF Detection Engine for further analysis.

3.3. Streamed Data Acquisition Module (SDM)

The SDM represents an enhanced version of the DAM, specifically designed to facilitate the distribution of multiple streams using N ring buffers established by the PF_RING module. When multiple rings are formed, each one is allocated to a specific CPU core. Packets arriving at a single network interface are then load-balanced across multiple streams, effectively distributing the load across various rings and CPU cores. To optimize stream balancing, SDM employs clustering to distribute traffic uniformly across multiple streams. In setups involving multiple streams, a cluster ID is assigned to each instance of the Detection Engine, with PF_RING sockets binding them to their corresponding cluster IDs. Consequently, each ring processes a distinct portion of the streams for different instances of the Detection Engine. Stream partitioning across clusters is managed through cluster policies, ensuring that all packets associated with the same stream are directed to the corresponding instance. The default policy for clustering is per-flow balancing as shown in Figure 3.

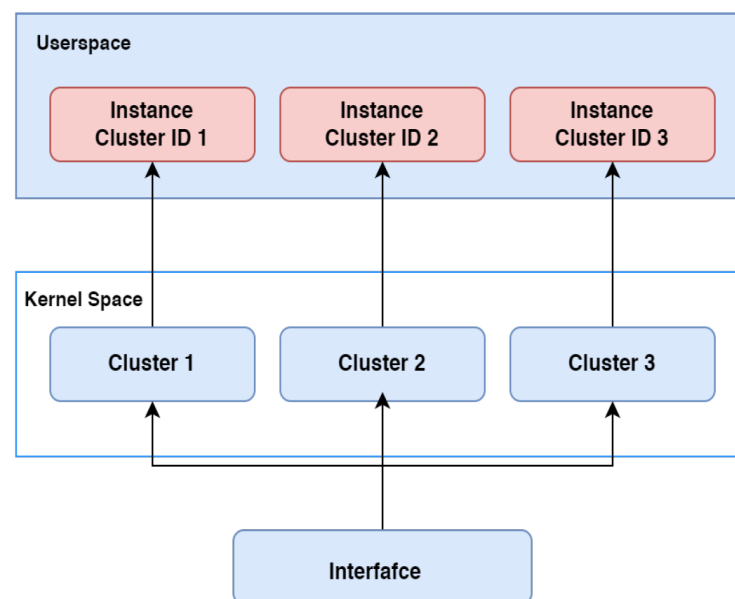


Figure 3. SDM Clustering.

Network packets in SDM Streams can be balanced using one of several cluster policies, depending on specific requirements. SDM can be configured with one of the following stream balancing algorithms to maintain traffic consistency and distribute the load among all CPU cores and associated ring buffers.

- Round robin;
- Two-tuple: src ip, dst ip;
- Four-tuple: src ip, src port, dst ip, dst port;
- Five-tuple: src ip, src port, dst ip, dst port, protocol.

3.4. Round Robin Stream Balancer

The round robin load balancer operates similarly to a first-come, first-served queuing system. It provides a distinct channel for each data stream, differentiated by the source and destination addresses. With this algorithm, data packets in an active stream are transferred to the iKern instances in a periodically repeating sequence. If a stream lacks data packets, the next stream takes its place, ensuring that link resources remain active at all times. The Round Robin balancer achieves max-min fairness when the packets within a stream are of equal size, prioritizing the stream that has been waiting for the longest. This algorithm distributes the streams across balanced flows without accounting for whether all involved instances have equal capacity to handle the load as shown in Figure 4.

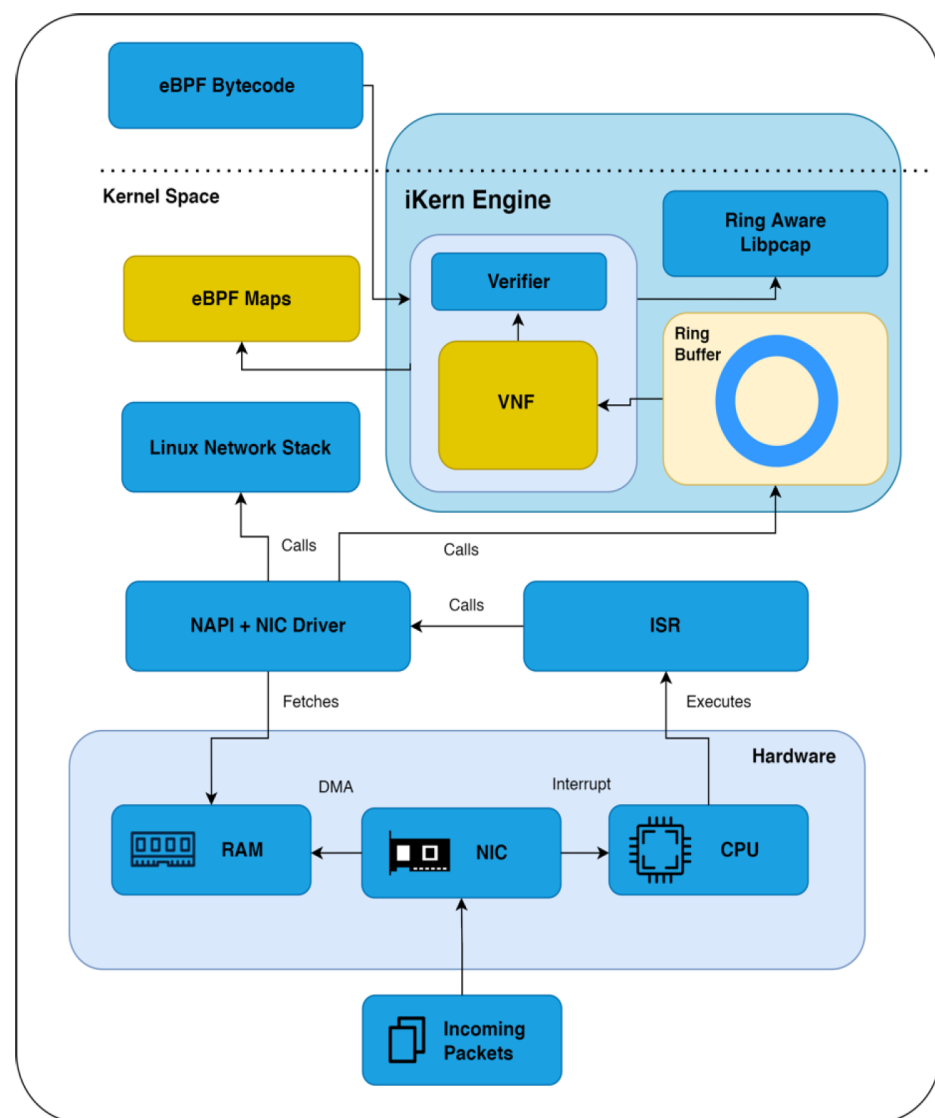


Figure 4. iKern Engine and DM.

3.4.1. Two-Tuple Stream Balancer

This load balancer, often referred to as IP Affinity, distributes data streams among multiple iKern detection engine instances based solely on the source and destination IP addresses of the packets. Using this method, packets from a specific IP are linked to a particular iKern instance through the clusters that have been created. Consequently, packets from specified IPs in the streams are directed to their associated instance as determined by

the cluster-ID. This approach is particularly effective when the IP addresses of packets in data streams remain consistent.

3.4.2. Four-Tuple Stream Balancer

The four-tuple balancer utilizes a combination of header fields from incoming packets to generate a hashing key scheme. This algorithm employs source IP, destination IP, source port, and destination port as the header fields for stream balancing. Network traffic distributed across multiple iKern instances is load-balanced using a hash function based on these header fields. Notably, the protocol type is not considered in generating the hash. Packets that share the same source IP, destination IP, source port, and destination port are consistently directed to a specific iKern instance.

3.4.3. Five-Tuple Stream Balancer

The SDM can be configured for load balancing using the five-tuple stream balancer. This algorithm utilizes a hash based on source IP, source port, destination IP, destination port, and protocol type to map packets to the available iKern instances. The key advantage of employing the five-tuple stream balancer lies in its ability to maintain consistency within a transport session. Packets that arrive at the SDM buffers and belong to the same TCP or UDP session are directed to the same cluster, ensuring session stickiness at the load-balanced endpoint. The use of the five-tuple scheme facilitates heavy load balancing due to the per-flow balancing technique, which helps preserve instance logic. Consequently, each iKern instance receives a subset of the packets, maintaining the consistency of traffic handling across the system.

3.5. In Kernel Detection Module (iKern)

iKern is meticulously designed and integrated within the Linux kernel to detect volumetric and multi-vector floods through its specialized detection algorithm and associated drop rules. It is directly connected to the DM for receiving network packets captured by the PF RING socket. The architecture of iKern comprises multiple modules functioning within the Linux kernel. The Network API (NAPI) facilitates the transfer of packets from the Network Interface Card (NIC) to the circular buffer.

Incoming packets are examined within the iKern Engine by the eBPF VNF, which is established by injecting eBPF bytecode from user space. This injected iKern detection algorithm and drop rules are stored in eBPF maps. Each time new eBPF bytecode is added to the Linux kernel, it undergoes scrutiny by a verifier to ensure compatibility and check for any syntax errors, thus safeguarding the kernel space state. Once verified, the bytecode, including the iKern algorithm and drop rules specifically crafted to identify attacks, is stored in eBPF maps.

Traffic that reaches the ring buffer is checked against the attributes stored in the eBPF maps. If any malicious activity is detected, the iKern drop rule is activated to discard the offending packets and trigger an alert to user space. Traffic that passes from the VNF to the ring buffer-aware libpcap is thus cleansed of any volumetric or multi-vector floods. The internal architecture of iKern Detection, along with the Data Acquisition Module, is depicted in the figure.

iKern Detection Algorithm (IDA) and Drop Rules (IDR): The iKern detection algorithm Algorithm 1 scrutinizes incoming traffic to determine whether it is malicious or benign. Should it detect a volumetric or multi-vector attack, iKern categorizes the threat based on its severity, labeling it as either a high-priority or low-priority attack. In response, it sends alerts that include the attacker's address, port number, and the type of attack. A high-volume flood is identified and classified as a high-priority attack if the following condition is met;

$$\delta t(Rp) < \beta \times dt \quad (1)$$

In Equation (1), Rp represents the time difference between the first and last packets received during the fixed detection interval, dt . β is a small positive number such that $0 < \beta < 1$.

If the condition in the equation is not met, the IDA categorizes the attack as a Low-priority attack. The enhanced iKern detection algorithm integrates traditional threshold-based detection methods with advanced ML techniques to improve the accuracy and responsiveness of DDoS attack detection. The algorithm initializes essential variables including a packets counter and a detection time window, continuously monitoring the network traffic within this period. If the packets count exceeds a predefined threshold, the algorithm calculates the time difference between successive packets to determine the urgency of the traffic flow. A condition based on this time difference and a detection factor decides if a high-priority attack alert should be issued, indicating a potential volumetric attack.

Algorithm 1 iKern Detection Algorithm (IDA)

```

1: Threshold value  $\leftarrow$  thresh_value
2: Packets counter  $\leftarrow$  Pc
3: Detection time  $\leftarrow$  dt
4: Detection factor  $\leftarrow \beta$ 
5: Machine Learning Model  $\leftarrow$  ML_model
6: if detection time (dt) not expired then
7:   while packet received do
8:     Pc  $\leftarrow$  Pc + 1
9:     if Pc > thresh_value then
10:       $\Delta t \leftarrow$  calculate time difference
11:      if  $\Delta t < \beta \times dt$  then
12:        Send High-priority attack alert to iKern node
13:      else
14:        Invoke ML_model with packet features
15:        Classify packet
16:        Send result to iKern node
17:      end if
18:    end if
19:  end while
20: else
21:   Reset timer
22:   Pc  $\leftarrow$  0
23: end if

```

Alternatively, when the packet flow does not immediately suggest a high-priority threat, the features of the packets are analyzed using an ML model. This model classifies the traffic based on learned patterns, which helps in distinguishing between normal, low-priority, and high-priority DDoS attacks. The result of this classification is then communicated to the iKern node, ensuring that each detected traffic pattern is appropriately addressed. This integration of ML allows the iKern system to adapt to changing attack tactics and enhances its capability to secure the network against a diverse range of threats.

The iKern detection engine captures and stores the IP addresses, port numbers, protocols, and types of suspected attacks, subsequently initiating the iKern attack mitigation procedure. When high-priority attack packets are identified, they are immediately dropped and the corresponding IP addresses are added to the eBPF blacklist filter; this action is promptly alerted. Conversely, IP addresses associated with low-priority attacks are directed to the graylist filter and are temporarily blocked to mitigate potential threats.

Legitimate IP addresses can be added to the iKern whitelist filter by users. This inclusion helps bypass the inspection of packets from trusted sources, ensuring fluid network operations for recognized entities. The attack detection process utilizes the iKern detection algorithm in conjunction with eBPF filters. The IDR employs eBPF actions to sift through incoming traffic flagged by the IDA. Malicious packets identified by their IP addresses in the blacklist are preemptively dropped using eBPF IP or protocol filters. Additionally, the IP addresses of detected malicious packets are cataloged into the graylist

or blacklist prior to being added to the eBPF maps. These actions, crucial to maintaining network integrity, are outlined in the iKern Drop Rules algorithm within Algorithm 2 IDR.

Algorithm 2 iKer Drop Rules Algorithm (IDR)

```

1: if warning packet received then
2:   if in GrayList then
3:     Move to Blacklist;
4:   else if high-priority attack alert then
5:     Add to GrayList;
6:     Set High-priority active timer;
7:   else
8:     Add to GrayList;
9:     Set Low-priority active timer;
10:    Set expiration timer for false positives;
11:  end if
12: else ▷ Handling Data Packets
13:   if in WhiteList then
14:     Forward packet;
15:   else if in GrayList and (destination address is victim) and (any active timer) then
16:     Drop packet;
17:   else if in BlackList then
18:     Drop packet;
19:   else
20:     Forward packet to user space;
21:   end if
22: end if
23: Analyze historical data to adjust blacklist and threat levels;
24: Update threat levels based on real-time traffic analysis;

```

4. Experiments and Results

The detailed results and evaluation of the proposed iKern-IDPS engine are presented, starting with an initial phase of testing that focuses on evaluating the iKern DM and comparing it with default libraries. These tests are conducted using identical parameters for both the tested libraries and the iKern DM. Packet reception and load balancing across multiple instances are facilitated by integrating the PF_RING socket into the iKern DM. The experiments demonstrate various load-balancing techniques applied to efficiently manage large streams of volumetric and multi-vector attacks. These attacks employ multiple vectors, continuously varying the rate and type of floods launched against a system. Also, the characteristics of the testbed environment are presented in Table 2.

Table 2. Summary of system-level deployment.

Component	Specification
Processor	Intel Xeon CPU E5-2630 v4 @ 2.20 GHz, 10 cores
RAM	64 GB DDR4
Storage	1 TB SSD
Operating System	Ubuntu 20.04 LTS
Network Interfaces	10 Gbps Ethernet NIC
Data Acquisition Modules	PF_RING enhanced with custom modules
Software Tools	Customized PF_RING, eBPF for kernel-level packet filtering
Attack Simulation Tools	HULK, SLOWLORIS, LOIC, RUDY

Furthermore, the capability of the system to detect and mitigate such variable stream properties is rigorously tested. The detection accuracy and the iKern acquisition capacity are evaluated using specified testing parameters across different categories of attacks. Additionally, experiments to assess the reception of large-scale floods are performed using the iKern DM, showcasing its effectiveness in handling extreme network loads.

Figure 5 illustrates the network setup and the various components of iKern involved in data acquisition, attack generation, and the detection process. The setup includes multiple virtual machines (VMs), each configured to launch floods at a destination IP address where iKern is deployed. The first VM is designated for launching volumetric floods using LOIC and HOIC attack generation tools. The second VM employs HULK for initiating floods, while the third VM is tasked with replaying CICDDOS2019 pcap files using tcpreplay. These attacks are captured by the iKern Data Acquisition Module (DM) and subsequently forwarded to the iKern Detection Module. This module is responsible for identifying and eliminating malicious packets and floods, ensuring robust defense against these threats.

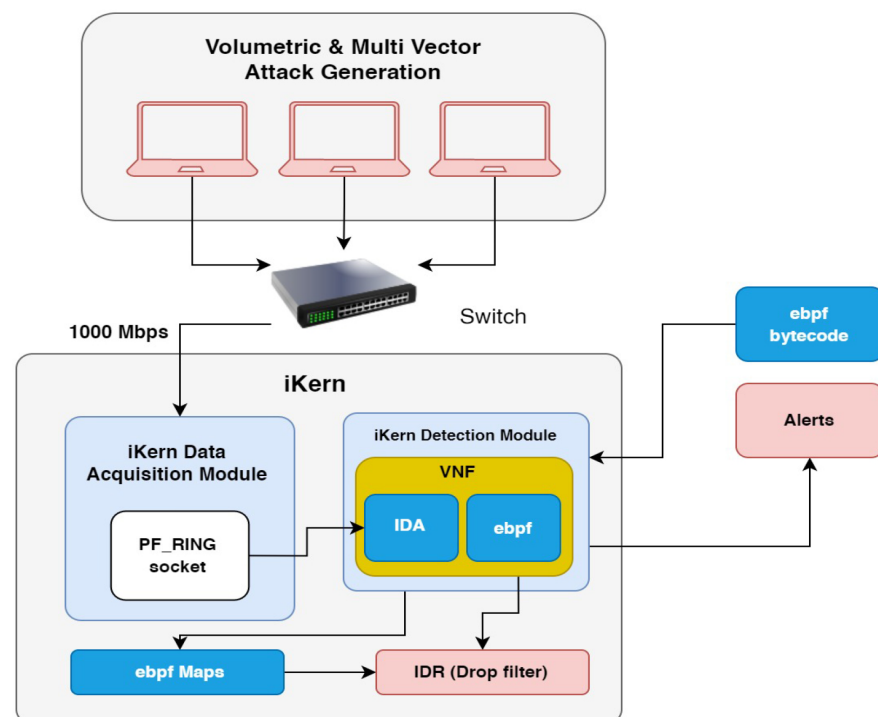


Figure 5. iKern network setup.

4.1. iKern—Data Acquisition Module (DM)

To assess the efficiency of the iKern DM, the default Linux packet-capturing libraries were tested and the results were compared with those of the iKern DM.

4.1.1. Libpcap

This test was conducted to evaluate the reception capacity and maximum threshold of libpcap. Floods involving multiple packet sizes and throughput rates were launched using HOIC and HULK. At an 84 Mbps throughput, libpcap achieved a reception rate of approximately 99% for smaller packets. Packets with an average size of 512 bytes had a reception rate of 98.5%. Packet sizes of 1500 bytes, which correspond to the MTU of TCP packets, exhibited a drop rate of around 4%. When a jumbo frame flood was launched at 84 Mbps, libpcap successfully received 91% of the packets as shown in Figure 6. Further results indicated the reception rates of these packets at higher throughputs of 168 Mbps, 252 Mbps, and 336 Mbps. Libpcap began dropping smaller packets when the throughput exceeded 120 Mbps. At 168 Mbps, it received 97% and dropped 3% of small packets of 64 bytes. For packets of average size, 96% were received and the drop rate increased to

7% for 1500 Mbps packet size at the same throughput. The drop rate for jumbo frames increased to 14%, and at a 336 Mbps flood, 27% of jumbo frames were dropped.

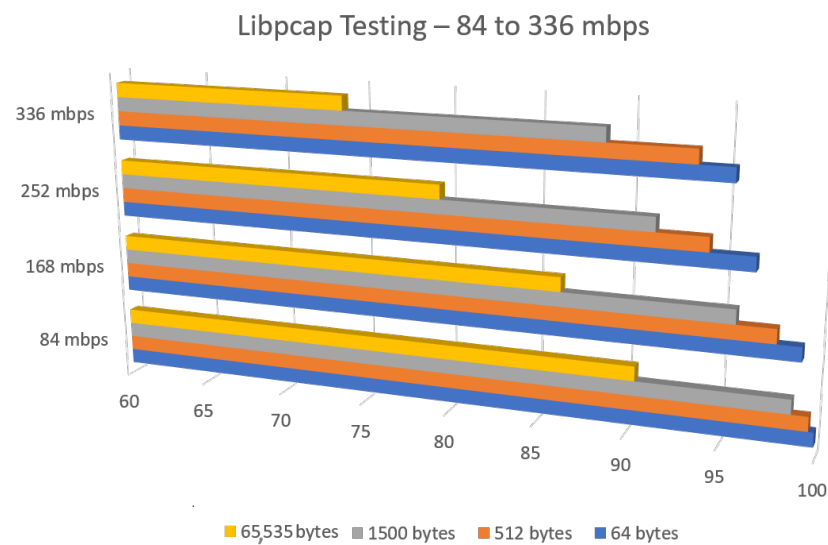


Figure 6. Libpcap—84 to 336 Mbps.

4.1.2. AF_PACKET

To assess the maximum threshold of AF_PACKET, tests were conducted under similar conditions, evaluating the reception rate at multiple data rates ranging from 84 Mbps to 336 Mbps. Volumetric floods were generated using HOIC and HULK to examine the variable packet size reception rates. In comparison to libpcap, AF_PACKET demonstrated a 100% reception rate at 84 Mbps for packet sizes ranging from 64 bytes to 1500 bytes, although it dropped 4% of jumbo frames as shown in Figure 7.

During a 168 Mbps flood, AF_PACKET achieved 100% reception for 64-byte packets and received 97.5% of average-sized packets. At a 336 Mbps flood rate, the reception of 64-byte packets dropped to 4%, 512-byte packets to 6%, and MTU packets to 9%. Jumbo frames experienced a 20% drop rate using AF_PACKET at 336 Mbps.

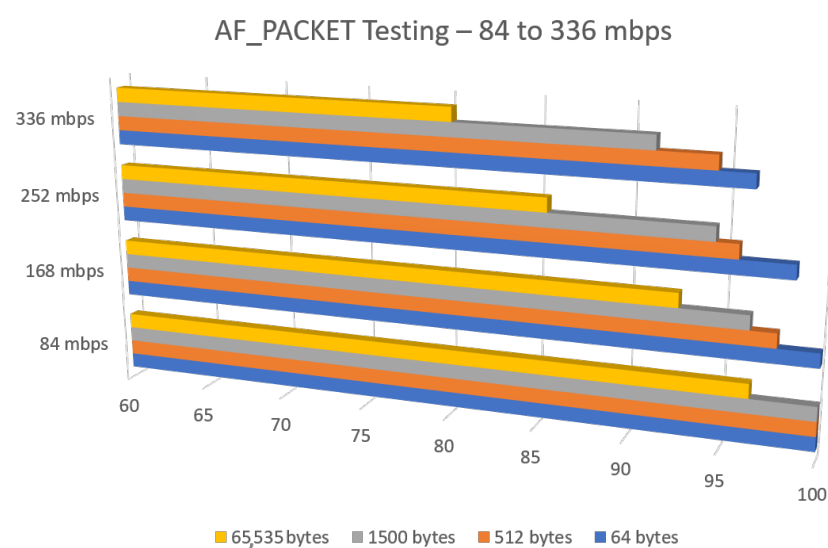


Figure 7. AF_PACKET—84 to 336 Mbps.

4.1.3. iKern DM Using PF_RING Socket

The iKern Data Acquisition Module, integrated with a PF_RING socket, was tested under the same parameters as those used for libpcap and afpacket. The tests involved volumetric flood throughput ranging from 84 Mbps to 445 Mbps, utilizing HOIC and HULK. The packet sizes tested varied from 64 bytes to 1500 bytes, with jumbo frame sizes of 65,535 bytes.

The iKern DM exhibited a 100% reception rate from 84 Mbps to 168 Mbps across all packet sizes, including jumbo frames. However, it encountered a 2% drop rate in jumbo frame flood packets at a 252 Mbps throughput. At 336 Mbps, packets ranging from 64 bytes to 1500 bytes were received with a 0% drop rate, while jumbo frames experienced a 3% drop rate. Further testing at 445 Mbps was conducted to evaluate the reception rate of average packet sizes and the maximum threshold of iKern DM. This test showed a 100% reception rate for 64- and 512-byte packet floods, a 3% drop rate for 1500-byte packets, and approximately a 4% drop rate for jumbo frames as shown in Figure 8.

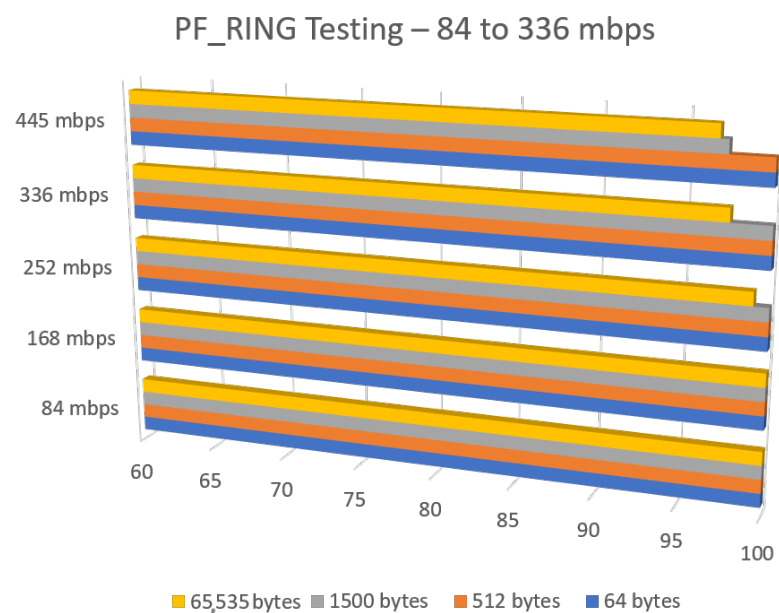


Figure 8. PF_RING—84 to 445 Mbps.

The settings for these experiments included a minimum number of slots in the PF_RING socket set at 4096, with transparent mode disabled, TX capture enabled, and no CPU binding. These configuration details are documented in Table 3. This comprehensive testing demonstrates the robust performance and reliability of the iKern DM under various network load conditions.

Table 3. PF_RING Socket Parameters—Not Optimized.

Parameter	Value
Min num slots	4096
TX Capture	default
Transparent Mode	default
CPU binding	Not bound
Load Balancing	Not balanced

Figure 9 illustrates the total number of packets received by the iKern Data Acquisition Module (DM) utilizing the PF_RING socket. The data correspond to a throughput of approximately 445 Mbps and includes both small and average packet sizes.


```

Total Stats:
[ total pkts:1'824'389 ]
[ pkts dropped:0 ]
[ 0.0% dropped ]
[1'824'389 pkts rcvd][334'727'934 bytes rcvd] [ Latency : 0.000004 ]
[ pfiring running time: 6.000451 sec]
[304'041.97 average pkt/sec][Data Rate: 446.27 Mbit/sec]

*****
This Instance Stats: [244'891 pkts rcvd][1'000.09 ms][244'868.22 pps][0.44 Gbps]
*****

+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+
Total Stats:
[ total pkts:2'083'141 ]
[ pkts dropped:0 ]
[ 0.0% dropped ]
[2'083'141 pkts rcvd][390'538'165 bytes rcvd] [ Latency : 0.000004 ]
[ pfiring running time: 7.000562 sec]
[297'567.68 average pkt/sec][Data Rate: 446.29 Mbit/sec]

*****
This Instance Stats: [258'752 pkts rcvd][1'000.11 ms][258'723.28 pps][0.45 Gbps]
*****

+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==

```

Figure 9. iKern DM—445 Mbps.

4.2. iKern DM Using PF_RING Socket on 1 Gbps Throughput

The iKern DM was tested on three CPU cores by deploying three instances of iKern to evaluate the volumetric flood reception capacity at a 1 Gbps data rate. Initial findings revealed that a single core operating a lone iKern instance resulted in significant packet loss: approximately 30% of 64-byte packets, 40% of average-sized packets, and about 45% of 1500-byte packets were dropped. Furthermore, only 50% of jumbo frames were successfully received using a single core and a single PF_RING socket as shown in Figure 10.

```

[210'649.36 average pkt/sec][Data Rate: 1'034.65 Mbit/sec]

*****
This Instance Stats: [189'162 pkts rcvd][1'000.06 ms][189'149.13 pps][0.93 Gbps]
*****

+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==+==
Total Stats:
[ total pkts:1'873'607 ]
[ pkts dropped:0 ]

```

Figure 10. iKern DM—445 Mbps.

In a subsequent experiment, a notable improvement in reception rate was achieved by utilizing two cores, each running an iKern DM instance with individual PF_RING sockets for packet reception. At a 1 Gbps throughput generated by HOIC and HULK, this setup successfully received 100% of packets ranging from 64 to 512 bytes. However, there was a 1% drop observed in 150-byte packet size and approximately 10% of jumbo frames were lost due to buffer overflow.

The third experiment expanded the setup to three cores, each hosting an iKern instance with individually created PF_RING sockets. This configuration was intended to manage 1 Gbps of incoming traffic. Although the CPU overhead was not evenly distributed across the three cores, leading to increased overall CPU load, the reception was highly efficient. The data showed that 100% of packets of all sizes were received by the three iKern instances at 930 Mbps. This series of experiments underscores the scalability and effectiveness of the iKern DM when additional hardware resources are utilized, albeit with considerations regarding CPU overhead and distribution.

PF_RING socket optimization was applied to the experiment using the parameters outlined in Table 4. This table provides a detailed breakdown of the adjustments made to optimize the socket configuration, thereby enhancing the handling of high-throughput data rates and volumetric floods across multiple iKern instances. The same experiment

was conducted using the default parameters of the PF_RING socket at a 1 Gbps volumetric flood rate, with multiple packet sizes, across three iKern instances. Refer to Figure 11 for details. In this experiment, packet drops were observed with the 1500-byte packet floods, where approximately 2% of the packets were dropped at a throughput of 1 Gbps. Additionally, 8% of jumbo frames were also dropped when the iKern DM had not undergone optimization procedures.

Table 4. PF RING Socket Parameters—Optimized.

Parameter	Value
Min num slots	16,384
TX Capture	0
Transparent Mode	1
CPU binding	Bound
Load Balancing	Round Robin

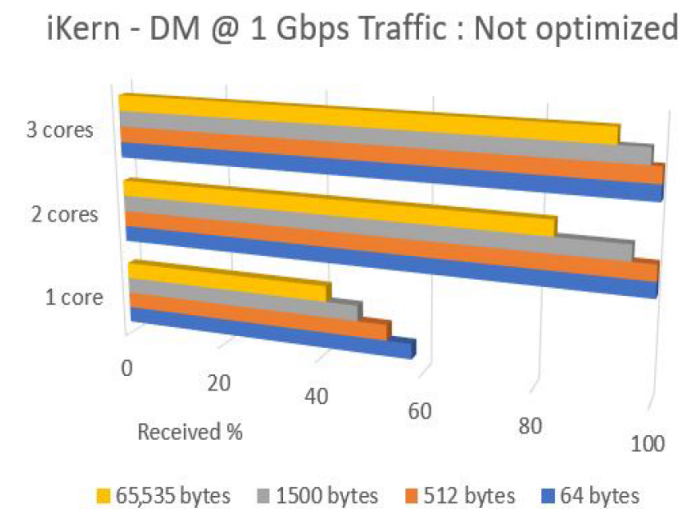


Figure 11. iKern DM—without optimization.

4.3. iKern DM Load Balancer

In this experiment, to evaluate the efficiency of handling an incoming flood across all three iKern instances, the iKern DM load balancer was integrated and its performance was assessed by comparing the results with and without the use of a load balancer. Three distinct load balancing techniques—round robin, two-tuple, and four-tuple balancers—were implemented to test the CPU overhead and load distribution among the three iKern instances. Volumetric floods were generated using HOIC and HULK at a rate of 1 Gbps. While the reception rate among all three cores reached 100%, variations in load balancing efficiency were observed with the different techniques employed as shown in Figure 12.

This experiment demonstrates that during the volumetric flood initiated by HOIC and HULK, random source IP addresses were utilized to simulate the real-world conditions of volumetric and multi-vector flood attacks. Among the load-balancing techniques tested, round robin proved to be the most effective. It distributed the workload almost evenly across all three cores, a result attributable to the inherent nature of the round robin distribution. Each of the three iKern instances was bound to an individual core, and socket clustering was employed to prevent packet collisions, ensuring that each instance exclusively processed its assigned packets. The parameters used in this load balancing experiment were consistent with those in the iKern DM test involving load balancing, as detailed in Table 5.

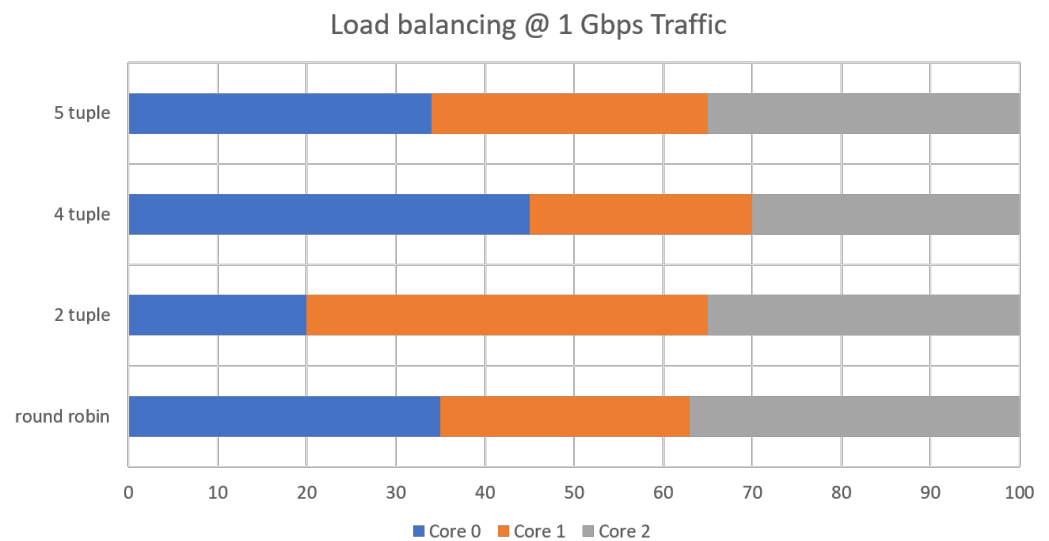


Figure 12. iKern DM—load balancing on 3 cores.

Table 5. PF_RING Multiple Sockets—Optimized.

Parameter	Value
Min num slots	16,384
TX Capture	0
Transparent Mode	1
CPU binding	Bound
Load Balancing	Round Robin

4.4. iKern DM CPU Utilization

In this experiment, CPU utilization was assessed under conditions without load balancing and CPU binding, and the same test was conducted with load balancing and CPU binding to evaluate the overall CPU utilization across three iKern instances at a 1 Gbps throughput. The CPU utilization was distributed among all three cores. The results indicate that, without CPU binding and load balancing, the processing of a 64-byte packet flood across the three cores showed a significant disparity in CPU utilization percentages: core 0 was utilized at approximately 39%, core 1 at around 30%, and core 2 at 59%. A similar variation in CPU utilization among the three cores was observed across all packet-size floods as shown in Figure 13.

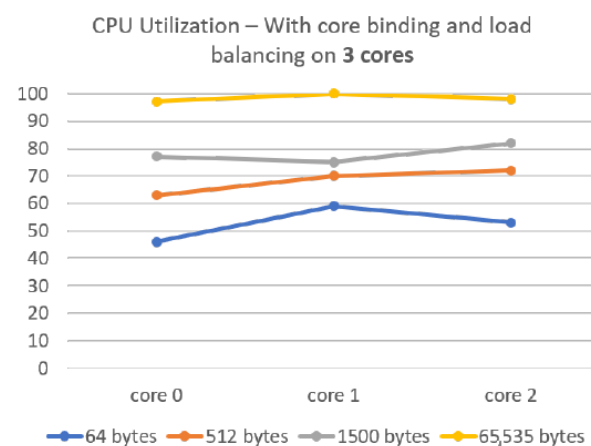


Figure 13. iKern DM—CPU utilization without load balancing.

By implementing CPU binding and load balancing in the same experiment, it was observed that the workload across all three iKern instances was nearly evenly distributed among the three CPU cores. For example, during a 64-byte flood test, CPU utilization was measured at approximately 48% on core 0, 59% on core 1, and 52% on core 2 as shown in Figure 14. This configuration demonstrated a more balanced CPU utilization across cores compared to previous experiments, highlighting the effectiveness of using CPU bind and load balancing techniques to manage resource allocation more efficiently.

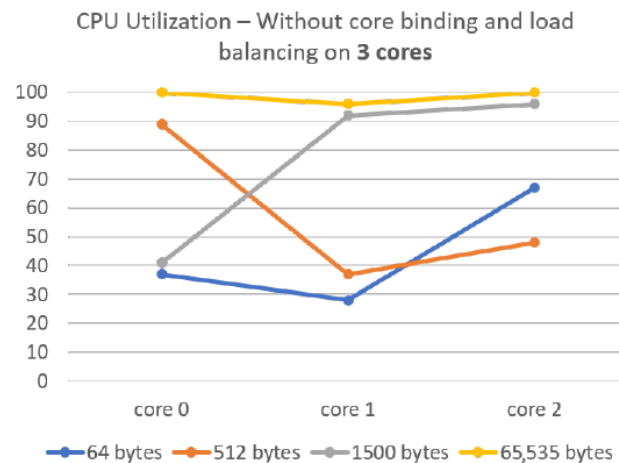


Figure 14. iKern DM—CPU utilization with load balancing.

4.5. iKern Engine—Volumetric and Multi-Vector Flood Detection

In this section, the performance of the iKern Engine in detecting volumetric and multi-vector flood attacks is tested and evaluated, focusing on the accuracy of the iKern engine utilizing eBPF technology. This experiment involved testing at variable data rates on a single stream, incorporating different attack vectors. Multiple attack vectors were generated using tools such as HOIC, LOIC, RUDY, and HULK, while the CIC-DDoS2019 dataset was employed to simulate a variety of volumetric floods including UDP, TCP, PortMap, NetBIOS, LDAP, MSSQL, UDP-Lag, SYN, NTP, DNS, and SNMP floods at high data rates individually.

In the initial experiment designed to test the detection and mitigation capabilities against multi-vector floods, two specific attack vectors were employed to challenge the iKern's detection system. The attack involved generating a data rate of 1 Gbps, with the attack sustained continuously for 15 min as shown in Figure 15. The variable attack vectors used in this test were TCP SYN and UDP IP Fragment, providing a rigorous test of the iKern's response mechanisms as facilitated by the IDR using eBPF technology.

In the subsequent experiment, four attack vectors were launched at the iKern at a 1Gbps data rate to assess the capacity of the iKern Data Acquisition Module (DM) and the Intrusion Detection Algorithm (IDA). Over a continuous 15-min period, it was observed that the iKern DM successfully received 100% of the packets from three vectors, while UDP IP fragment packets were received at a rate of 98.7%. Utilizing eBPF, the IDR effectively dropped 98.9% of TCP SYN packets, 97.4% of volumetric UDP packets, and approximately 99.4% of DNS amplification packets. Additionally, 98.5% of the received UDP IP Fragmented packets were dropped as shown in Figure 16.

Volumetric floods were assessed utilizing the CIC-DDoS2019 dataset. This pcap file comprises various types of attacks, including UDP, TCP, PortMap, NetBIOS, LDAP, MSSQL, UDP-Lag, SYN, NTP, DNS, and SNMP floods. These were replayed using TCP replay to simulate maximum system throughput, achieving rates close to 1 Gbps. As volumetric floods typically increase CPU utilization, both the CPU performance statistics and the accuracy of the IDR system were closely monitored during these tests.

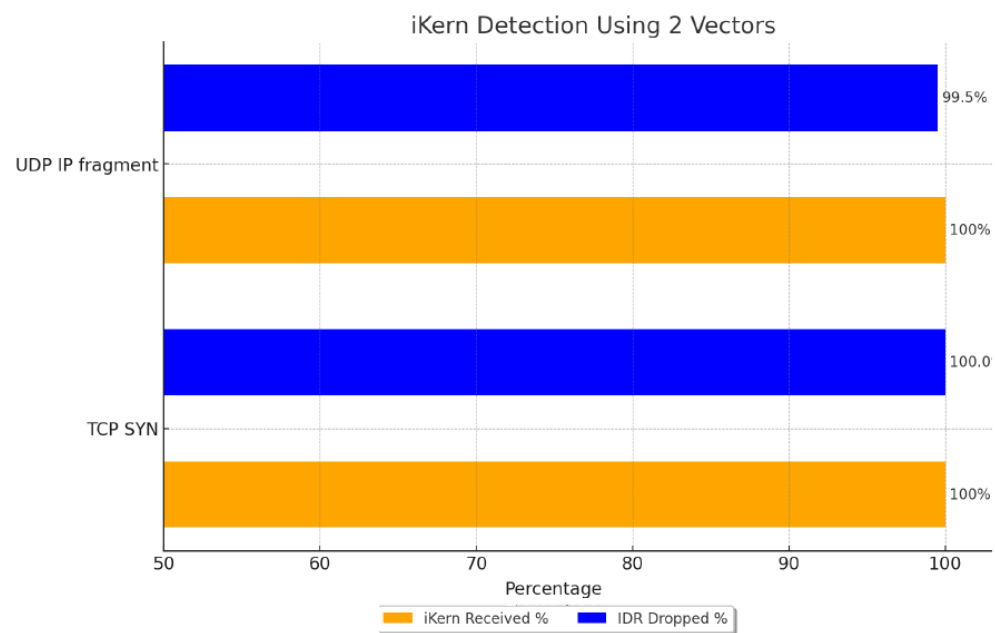


Figure 15. iKern detection and drop—2 vectors.

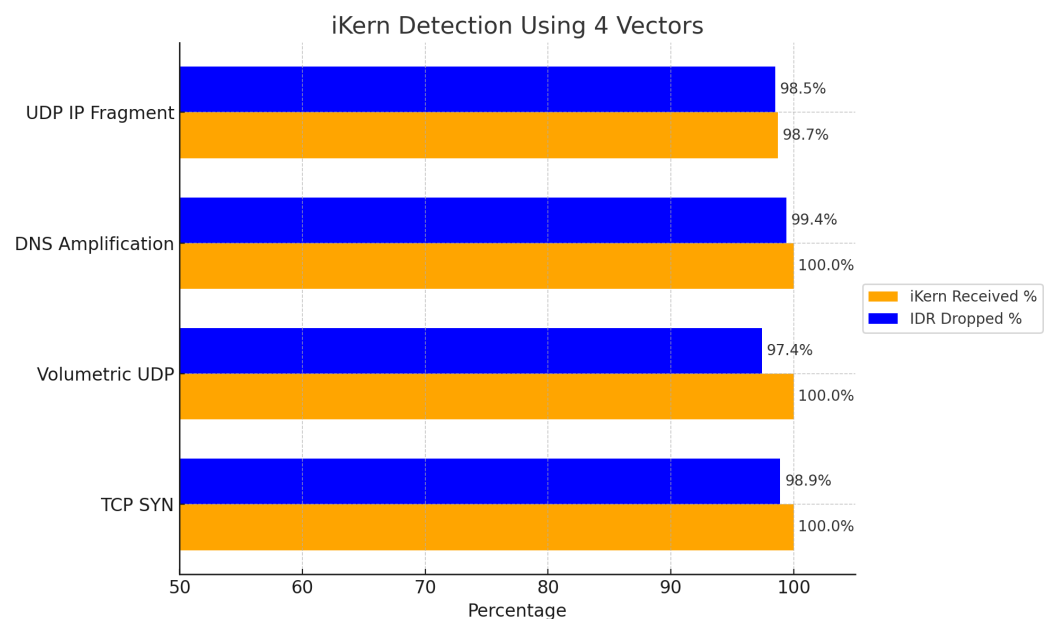


Figure 16. iKern detection and drop—4 vectors.

It was observed that using the CIC-DDoS2019 dataset to launch volumetric floods at a 1 Gbps data rate yielded 98.9% accuracy for UDP volumes, 97.4% for TCP volumes, 100% detection and drop rate for PortMap attacks, 100% accuracy for NetBIOS attacks, 99.7% for LDAP volumes, 97.5% for MSSQL attacks, 99% accuracy for SYN volume floods, 100% drop rate for DNS attacks, and 98.4% accuracy for SNMP volume floods. These floods were generated by replaying the pcap file containing the attacks using tcpreplay with the maximum throughput flag. The experiment achieved 100% packet reception at the iKern Data Acquisition Module without any packet loss due to buffer overflows or CPU overheads. The CPU utilization statistics for this experiment are depicted in Figure 17.

All three cores shared the load from the volumetric flood attack at a nearly identical percentage. The attack, which lasted 20 min, was carried out using a continuous loop replay of a pcap file. Throughout this period, the CPU utilization for each core was approximately 92% to 97%. Specifically, at the start of the attack (5 min in), the CPU utilization was

recorded at 95.2% for core 0, 97.1% for core 1, and 92.6% for core 2. By the end of the attack (20 min), the utilization was observed to be 97.3% for core 0, 92.7% for core 1, and 94% for core 2. This indicates that the load was evenly balanced across all three cores throughout the attack as shown in Figure 18.

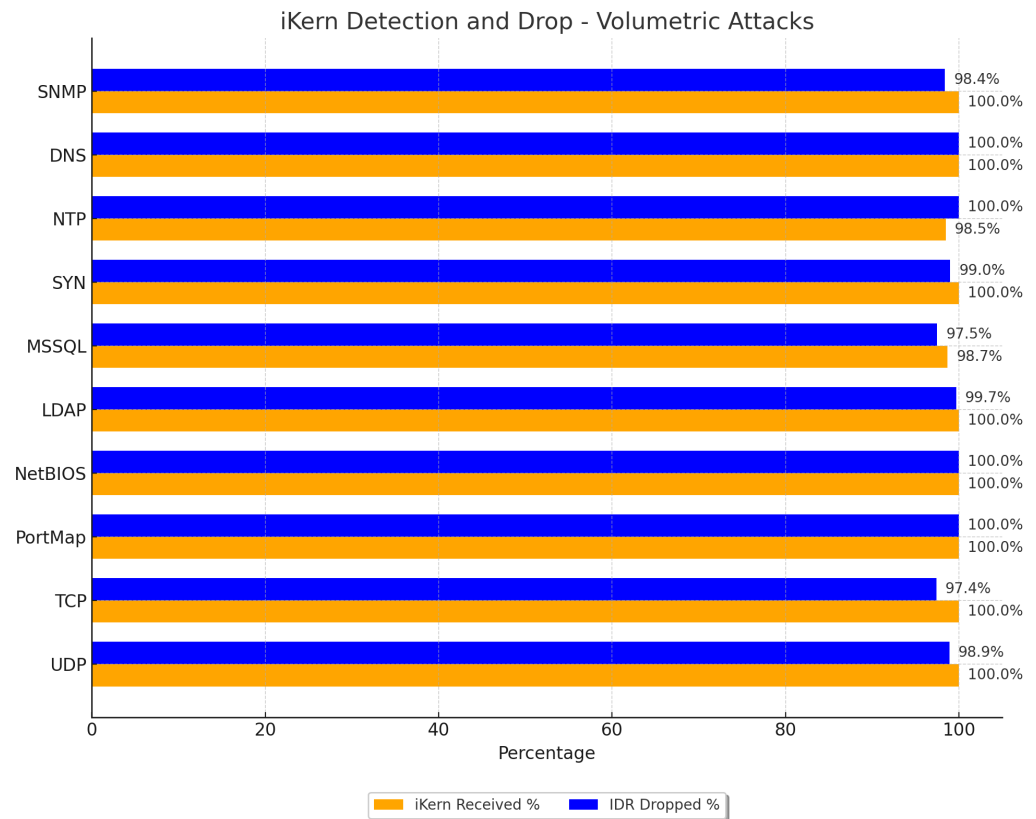


Figure 17. iKern detection and drop—volumetric attacks.

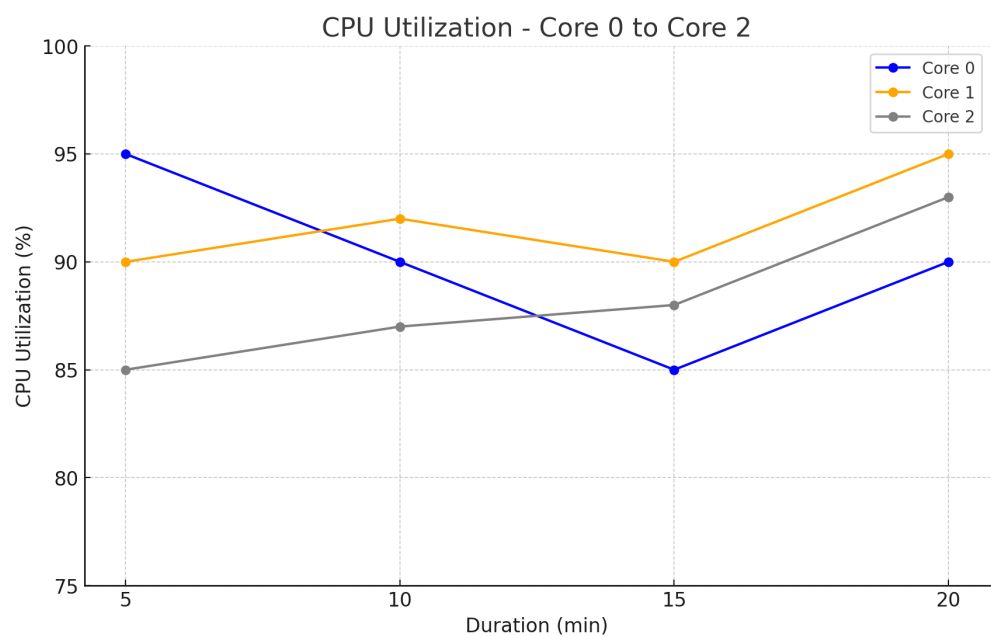


Figure 18. iKern detection and drop—volumetric attacks.

4.6. iKern vs. eBPF-Based VNF and IDPS

In the comparative analysis of iKern versus eBPF and IDPS, the iKern detection engine demonstrated a 98.84% accuracy in detecting volumetric and multi-vector floods. These floods were generated at a data rate of 1 Gbps using a network card that supports 1 Gbps. There was a 0% drop rate on all three cores used for acquiring the large floods, successfully forwarding all packets to the iKern detection engine. All three cores utilized up to 97% of the average CPU at 1 Gbps throughput. The iKern detection module, using eBPF and Streamed Data Acquisition, also showed 98.84% accuracy in detecting and mitigating the attacks.

Related work on eBPF-based Virtual Network Functions [14,15,17,21,28,33,36], used the afpacket module for capturing network packets, which performs acquisition using a ring buffer created inside the Linux kernel's default network stack. With afpacket, a packet reception rate of 420 Mbps was achieved for the launched volumetric and multi-vector floods. Detection performed using VNF within the kernel showed a 98.76% accuracy. Conversely, iKern achieved a throughput of 1000 Mbps using the pf_ring kernel socket under similar attack conditions. The IDA exhibited a 98.84% detection accuracy. iKern outperformed the related work in terms of acquisition of volumetric and multi-vector floods at high throughput, achieving a 0% packet drop rate and 100% packet reception at 1000 Mbps. The IDA excelled in the detection of floods, maintaining 98.84% accuracy. A load-balancing technique applied to the iKern DM distributed the load evenly among the three iKern instances.

Furthermore, IDPS data show no throughput or CPU usage reported, indicating that these systems might not have been deployed in a comparable environment or with similar metrics. Table 6 presents a comparative analysis of iKern and related work based on achieved throughput and detection accuracy of attacks, including the performance impact on traditional IDPS where applicable.

Table 6. iKern and related work performance comparison including traditional IDPS.

Title	VNF	DAQ	KDM/UDM	Mbps	CPU%	Acc%	IDPS Mbps	IDPS CPU%
eBPF-Based VNF [14]	ebpf	afpacket	KDM	420	100	98.76	X	X
Processing with eBPF [28]	ebpf	libpcap	KDM	178	100	97.42	X	X
Kernel-level trac [15]	ebpf	libpcap	KDM	120	95	98.2	X	X
iKern [33]	ebpf	pfRing	KDM	1000	97	98.84	Nil	Nil
IDPS [13]	libpcap	Nil	Nil	90	99.84	Nil	Nil	Nil

5. Conclusions

The rise in technology has increased the risk of sophisticated cyber threats that exploit network resources and monitoring systems, including modern IDS/IPS. These systems are vulnerable to multi-vector and volumetric flood attacks that consume system resources and exploit weaknesses in packet-capturing mechanisms, leading to buffer overloads and CPU overheads.

To address these modern threats and the need for efficient monitoring of large flows, the kernel-based traffic monitoring system iKern is proposed. iKern utilizes PF RING sockets to capture network packets at high data rates and employs an in-kernel Detection Engine to identify volumetric and multi-vector flood attacks before packets reach user space. Threats are mitigated using ebpf filters, with the detection algorithm injected into the Linux kernel using ebpf bytecode. Actions based on detected attacks are executed using ebpf maps to protect the system from incoming threats.

iKern has proven effective in detecting and mitigating multi-vector and volumetric floods, achieving 98.84% accuracy. During tests, iKern's Data Acquisition Module handled a data rate of 1Gbps with no packet loss, forwarding packets to the detection engine where they were analyzed and subsequently added to an IP blacklist in ebpf maps, ensuring

malicious packets were dropped before further inspection by the IDA during continuous pcap replay.

Future research could extend to managing network flows exceeding 1Gbps throughput. The multi-core SDM can be scaled using 10 Gbps multi-queue network interface cards. Additionally, this work could explore other cyber attacks, implementing their detection in kernel space using eBPF and VNFs. By shifting signature matching from user space to kernel space using VNFs, CPU overhead and extensive resource utilization could be significantly reduced.

Author Contributions: Conceptualization, M.A.A.; Methodology, Y.C.; Validation, Y.J.; Resources, N.A.; Data curation, F.B.H., M.A. and H.J.H.; Writing—original draft, Y.C. and M.A.; Funding acquisition, M.A.A. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by Prince Sultan University for paying the Article Processing Charges (APC) of this publication.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data presented in this study are available on request from the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Wang, S.Y.; Chang, J.C. Design and implementation of an intrusion detection system by using Extended BPF in the Linux kernel. *J. Netw. Comput. Appl.* **2022**, *198*, 103283. [\[CrossRef\]](#)
2. Høiland-Jørgensen, T.; Brouer, J.D.; Borkmann, D.; Fastabend, J.; Herbert, T.; Ahern, D.; Miller, D. The express data path: Fast programmable packet processing in the operating system kernel. In Proceedings of the 14th International Conference on Emerging Networking Experiments and Technologies, Heraklion, Greece, 4–7 December 2018; pp. 54–66.
3. Bertrone, M.; Miano, S.; Risso, F.; Tumolo, M. Accelerating linux security with ebpf iptables. In Proceedings of the ACM SIGCOMM 2018 Conference on Posters and Demos, Budapest, Hungary, 20–25 August 2018; pp. 108–110.
4. Freitas, E.; de Oliveira Filho, A.T.; do Carmo, P.R.; Sadok, D.F.; Kelner, J. Takeaways from an experimental evaluation of eXpress Data Path (XDP) and Data Plane Development Kit (DPDK) under a Cloud Computing environment. *Res. Soc. Dev.* **2022**, *11*, e26111234200. [\[CrossRef\]](#)
5. Latif, S.; Boulila, W.; Koubaa, A.; Zou, Z.; Ahmad, J. DTL-IDS: An optimized Intrusion Detection Framework using Deep Transfer Learning and Genetic Algorithm. *J. Netw. Comput. Appl.* **2024**, *221*, 103784. [\[CrossRef\]](#)
6. Ullah, S.; Boulila, W.; Koubaa, A.; Ahmad, J. MAGRU-IDS: A Multi-Head Attention-Based Gated Recurrent Unit for Intrusion Detection in IIoT Networks. *IEEE Access* **2023**, *11*, 114590–114601. [\[CrossRef\]](#)
7. Badotra, S.; Panda, S.N. SNORT based early DDoS detection system using OpenDaylight and open networking operating system in software defined networking. *Clust. Comput.* **2021**, *24*, 501–513. [\[CrossRef\]](#)
8. Bryant, B.D.; Saiedian, H. Improving SIEM alert metadata aggregation with a novel kill-chain based classification model. *Comput. Secur.* **2020**, *94*, 101817. [\[CrossRef\]](#)
9. Kizza, J.M. System intrusion detection and prevention. In *Guide to Computer Network Security*; Springer: New York, NY, USA, 2024; pp. 295–323.
10. Kamalov, F.; Moussa, S.; Khatib, Z.E.; Mnaouer, A.B. Orthogonal variance-based feature selection for intrusion detection systems. In Proceedings of the 2021 International Symposium on Networks, Computers and Communications (ISNCC), Dubai, United Arab Emirates, 31 October–2 November 2021; pp. 1–5. [\[CrossRef\]](#)
11. Yungaicela-Naula, N.M.; Vargas-Rosales, C.; Perez-Diaz, J.A.; Jacob, E.; Martinez-Cagnazzo, C. Physical assessment of an SDN-based security framework for DDoS attack mitigation: Introducing the SDN-SlowRate-DDoS dataset. *IEEE Access* **2023**, *11*, 46820–46831. [\[CrossRef\]](#)
12. Hu, Q.; Yu, S.Y.; Asghar, M.R. Analysing performance issues of open-source intrusion detection systems in high-speed networks. *J. Inf. Secur. Appl.* **2020**, *51*, 102426. [\[CrossRef\]](#)
13. Hadi, H.J.; Cao, Y.; Li, S.; Xu, L.; Hu, Y.; Li, M. Real-time fusion multi-tier DNN-based collaborative IDPS with complementary features for secure UAV-enabled 6G networks. *Expert Syst. Appl.* **2024**, *252*, 124215. [\[CrossRef\]](#)
14. Miano, S.; Risso, F.; Bernal, M.V.; Bertrone, M.; Lu, Y. A framework for eBPF-based network functions in an era of microservices. *IEEE Trans. Netw. Serv. Manag.* **2021**, *18*, 133–151. [\[CrossRef\]](#)
15. Caviglione, L.; Mazurczyk, W.; Repetto, M.; Schaffhauser, A.; Zuppelli, M. Kernel-level tracing for detecting stegomalware and covert channels in Linux environments. *Comput. Netw.* **2021**, *191*, 108010. [\[CrossRef\]](#)

16. Zhu, T.; Qiu, X.; Rao, Y.; Yan, H.; Zhou, Y.; Shi, G. HiAtGang: How to mine the gangs hidden behind DDoS attacks. *Chin. J. Electron.* **2022**, *31*, 293–303. [CrossRef]
17. Fuladi, R.; Baykas, T.; Anarim, E. The use of statistical features for low-rate denial-of-service attack detection. *Ann. Telecommun.* **2024**, 1–13. [CrossRef]
18. Saba, T.; Rehman, A.; Sadad, T.; Kolivand, H.; Bahaj, S.A. Anomaly-based intrusion detection system for IoT networks through deep learning model. *Comput. Electr. Eng.* **2022**, *99*, 107810. [CrossRef]
19. Kotey, S.D.; Tchao, E.T.; Gadze, J.D. On distributed denial of service current defense schemes. *Technologies* **2019**, *7*, 19. [CrossRef]
20. Gadze, J.D.; Bamfo-Asante, A.A.; Agyemang, J.O.; Nunoo-Mensah, H.; Opere, K.A.B. An investigation into the application of deep learning in the detection and mitigation of DDOS attack on SDN controllers. *Technologies* **2021**, *9*, 14. [CrossRef]
21. Sheeraz, M.; Durad, H.; Tahir, S.; Tahir, H.; Saeed, S.; Almuhaideb, A.M. Advancing Snort IPS to Achieve Line Rate Traffic Processing for Effective Network Security Monitoring. *IEEE Access* **2024**, *12*, 61848–61859. [CrossRef]
22. Vieira, M.A.; Castanho, M.S.; Pacifico, R.D.; Santos, E.R.; Júnior, E.P.C.; Vieira, L.F. Fast packet processing with ebpf and xdp: Concepts, code, challenges, and applications. *ACM Comput. Surv. (CSUR)* **2020**, *53*, 1–36. [CrossRef]
23. Viljoen, N.; Kicinski, J. Using eBPF as an Abstraction for Switching. 2018. Available online: http://vger.kernel.org/lpc_net2018_talks/eBPF_For_Switches.pdf (accessed on 22 June 2024).
24. Soldani, D.; Nahi, P.; Bour, H.; Jafarizadeh, S.; Soliman, M.F.; Di Giovanna, L.; Monaco, F.; Ognibene, G.; Risso, F. ebpf: A new approach to cloud-native observability, networking and security for current (5g) and future mobile networks (6g and beyond). *IEEE Access* **2023**, *11*, 57174–57202. [CrossRef]
25. Miano, S.; Bertrone, M.; Risso, F.; Bernal, M.V.; Lu, Y.; Pi, J. Securing Linux with a faster and scalable iptables. *ACM SIGCOMM Comput. Commun. Rev.* **2019**, *49*, 2–17. [CrossRef]
26. Ahmed, Z.; Alizai, M.H.; Syed, A.A. Inkev: In-kernel distributed network virtualization for dcn. *ACM SIGCOMM Comput. Commun. Rev.* **2018**, *46*, 1–6. [CrossRef]
27. Miano, S.; Doriguzzi-Corin, R.; Risso, F.; Siracusa, D.; Sommese, R. High-Performance Server-based DDoS Mitigation through Programmable Data Planes. In Proceedings of the 2019 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN), Dallas, TX, USA, 12–14 November 2019; pp. 1–6. Available online: <https://api.semanticscholar.org/CorpusID:164211962> (accessed on 22 June 2024).
28. Parola, F.; Risso, F.; Miano, S. Providing telco-oriented network services with ebpf: The case for a 5g mobile gateway. In Proceedings of the 2021 IEEE 7th International Conference on Network Softwarization (NetSoft). IEEE, Tokyo, Japan, 28 June–2 July 2021; pp. 221–225.
29. Xu, Q.; Wong, M.D.; Wagle, T.; Narayana, S.; Sivaraman, A. Synthesizing safe and efficient kernel extensions for packet processing. In Proceedings of the 2021 ACM SIGCOMM 2021 Conference, Virtual, 23–27 August 2021; pp. 50–64.
30. Zhu, Y.; Xu, M. Enhancing network throughput via the equal interval frame aggregation scheme for ieee 802.11 ax wlans. *Chin. J. Electron.* **2023**, *32*, 747–759. [CrossRef]
31. Canakci, B. Supporting Distributed Systems of Distributed Systems. Ph.D. Thesis, Cornell University, Ithaca, NY, USA, 2022.
32. Cassagnes, C.; Trestioreanu, L.; Joly, C.; State, R. The rise of eBPF for non-intrusive performance monitoring. In Proceedings of the NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium. IEEE, Budapest, Hungary, 20–24 April 2020; pp. 1–7.
33. Varghese, J.E.; Muniyal, B. An efficient ids framework for ddos attacks in sdn environment. *IEEE Access* **2021**, *9*, 69680–69699. [CrossRef]
34. Xu, Z.; Sun, Z.; Guo, L.; Muhammad, Z.H.; Chintia, T. Joint spectrum sensing and spectrum access for defending massive SSDF attacks: A novel defense framework. *Chin. J. Electron.* **2022**, *31*, 240–254. [CrossRef]
35. Alshathri, S.; El-Sayed, A.; El Shafai, W.; Hemdan, E.E.D. An Efficient Intrusion Detection Framework for Industrial Internet of Things Security. *Comput. Syst. Sci. Eng.* **2023**, *46*, 819–834. [CrossRef]
36. Bashah, N.S.K.; Simbas, T.S.; Janom, N.; Aris, S.R.S. Proactive DDoS attack detection in software-defined networks with Snort rule-based algorithms. *Int. J. Adv. Technol. Eng. Explor.* **2023**, *10*, 962.
37. AbdulRaheem, M.; Oladipo, I.D.; Imoize, A.L.; Awotunde, J.B.; Lee, C.C.; Balogun, G.B.; Adeoti, J.O. Machine learning assisted snort and zeek in detecting DDoS attacks in software-defined networking. *Int. J. Inf. Technol.* **2024**, *16*, 1627–1643. [CrossRef]
38. Arya, A.; Kumar, A.; Ahmad, S.S. DDoS Attack Detection Using Ensemble Machine Learning Approach. In Proceedings of the 2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT). IEEE, Delhi, India, 6–8 July 2023; pp. 1–5.
39. Hadi, H.J.; Hayat, U.; Musthaq, N.; Hussain, F.B.; Cao, Y. Developing Realistic Distributed Denial of Service (DDoS) Dataset for Machine Learning-based Intrusion Detection System. In Proceedings of the 2022 9th International Conference on Internet of Things: Systems, Management and Security (IOTSMS). IEEE, Milan, Italy, 29 November–1 December 2022; pp. 1–6.
40. Hadi, H.J.; Cao, Y.; Li, S.; Hu, Y.; Wang, J.; Wang, S. Real-Time Collaborative Intrusion Detection System in UAV Networks Using Deep Learning. *IEEE Internet Things J.* **2024**, Early Access. [CrossRef]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.